Trigger

HARSH KOTHADIYA

Part: 1

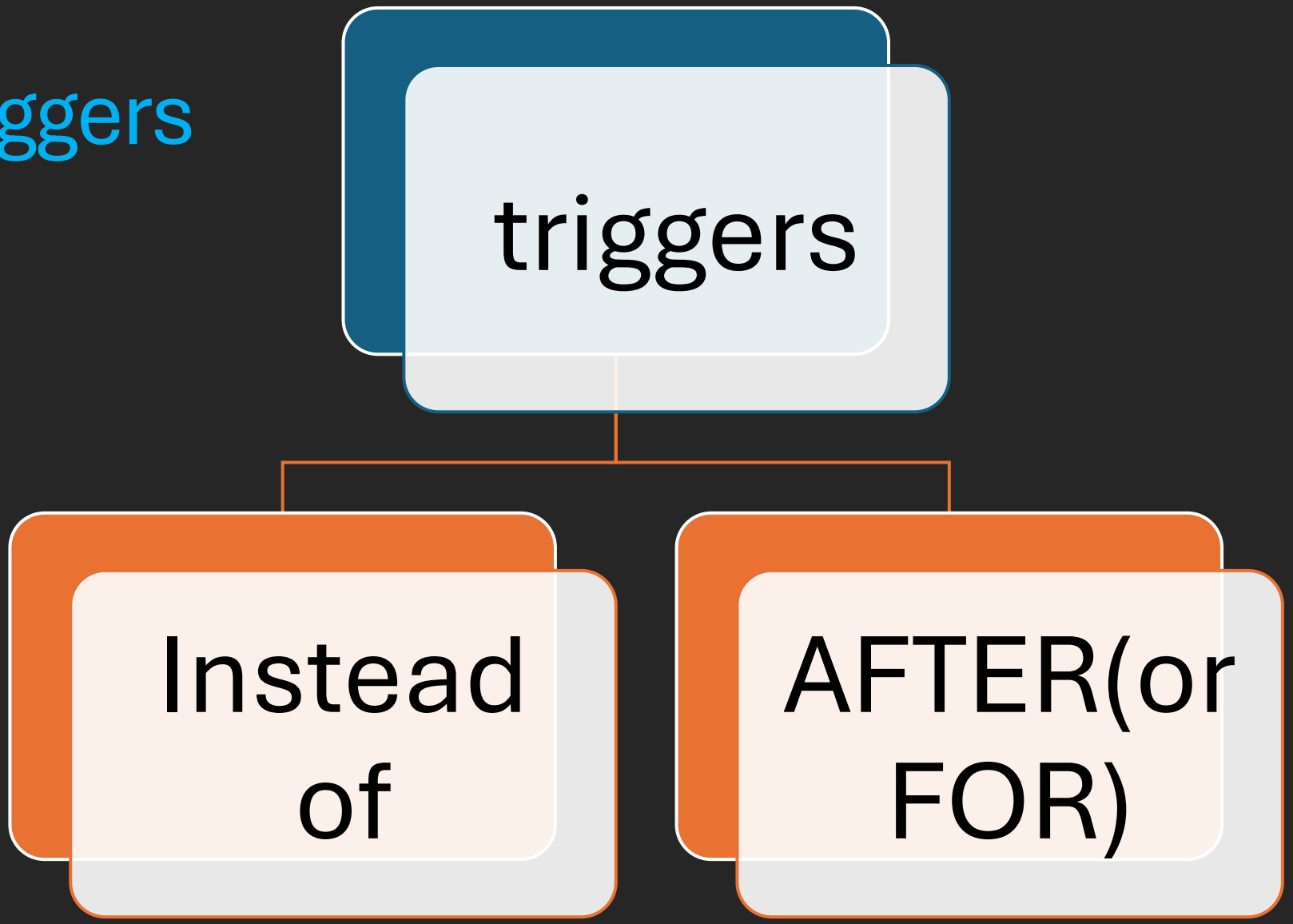
What is a Trigger?

- A Trigger is like an automatic action in SQL Server.
- Whenever something happens in a table (like INSERT, UPDATE, or DELETE), the trigger fires automatically and executes some code.
- Think of it like a "hidden rule" that runs when data changes.

Why Use Triggers?

- To enforce rules automatically (business logic).
- To log changes (keep history of updates/deletes).
- To validate data before saving.
- To cascade changes (update related tables).

two types of triggers



Instead of Triggers

- The Instead Of triggers are going to be executed instead of the corresponding DML operations.
- That means instead of the DML operations such as Insert, Update, and Delete, the Instead Of triggers are going to be executed.

After Triggers(FOR Triggers)

- The After Triggers fires in SQL Server execute after the triggering action.
- That means once the DML statement (such as Insert, Update, and Delete) completes its execution, this trigger is going to be fired.

🔹 Flow of INSTEAD OF Trigger

```pgsql
[User runs INSERT/UPDATE/DELETE]
                |
                v
   Normal Action is SKIPPED ❌
                |
                v
  INSTEAD OF Trigger Fires (custom logic)
```

The trigger **replaces** the action —

SQL Server won't run the default action unless

you code it inside the trigger.

🔹 Flow of AFTER Trigger

```pgsql
[User runs INSERT/UPDATE/DELETE]
                |
                v
     Normal Action Happens
                |
                v
       AFTER Trigger Fires
```

✅ The trigger runs **after** SQL Server completes the actual action.

Part: 2

```sql
--Trigger

--Step:1 -->Create database
CREATE DATABASE HK_Trigger;

--Step:2 -->Use database
USE HK_Trigger;
```

```sql
--Step:3 -->Create Table and insert data

CREATE TABLE Person(
    Person_Id INT,
    Person_Fname VARCHAR(10),
    Person_Lname VARCHAR(10),
    Person_Email VARCHAR(20)
);
INSERT INTO Person(Person_Id,Person_Fname,Person_Lname,Person_Email)
VALUES
    (1,'F-1','L-1','FL-1.@gmail.com'),
    (2,'F-2','L-2','FL-2.@gmail.com'),
    (3,'F-3','L-3','FL-3.@gmail.com');
SELECT * FROM Person;
```

```
22
23  SELECT * FROM Person;
```

No issues found

Results | Messages

Person_Id	Person_Fname	Person_Lname	Person_Email
1	F-1	L-1	FL-1.@gmail.com
2	F-2	L-2	FL-2.@gmail.com
3	F-3	L-3	FL-3.@gmail.com

Part: 3

```sql
--Step:4 INSTEAD OF Trigger
CREATE TRIGGER TR_Person_INSTEAD_OF_INSERT
ON Person
INSTEAD OF INSERT
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO Person(Person_Id,Person_Fname,Person_Lname,Person_Email)
    VALUES (500,'F-500','L-500','FL-500.@gmail.com')
END
GO
```

```sql
38 SELECT * FROM Person;
```

9 0

Results | Messages

Person_Id	Person_Fname	Person_Lname	Person_Email
1	F-1	L-1	FL-1.@gmail.com
2	F-2	L-2	FL-2.@gmail.com
3	F-3	L-3	FL-3.@gmail.com

```sql
--Step:5 Normal Insert
    INSERT INTO Person(Person_Id,Person_Fname,Person_Lname,Person_Email)
    VALUES (5,'F-5','L-5','FL-5.@gmail.com')

SELECT * FROM Person;
```

Results | Messages

	Person_Id	Person_Fname	Person_Lname	Person_Email
1	1	F-1	L-1	FL-1.@gmail.com
2	2	F-2	L-2	FL-2.@gmail.com
3	3	F-3	L-3	FL-3.@gmail.com
4	500	F-500	L-500	FL-500.@gmail.com

--here F-5 not insert because INSTEAD OF Trigger fire

```sql
--INSTEAD OF Trigger use for UPDATE
CREATE TRIGGER TR_Person_INSTEAD_OF_UPDATE
ON Person
INSTEAD OF UPDATE
AS
BEGIN

    SET NOCOUNT ON;

    UPDATE Person
    SET Person_Fname = 'Harsh'
    WHERE Person_Id = 1;

END
GO
```

```sql
SELECT * FROM Person;
```

Person_Id	Person_Fname	Person_Lname	Person_Email
1	F-1	L-1	FL-1.@gmail.com
2	F-2	L-2	FL-2.@gmail.com
3	F-3	L-3	FL-3.@gmail.com
500	F-500	L-500	FL-500.@gmail.com

```sql
--normal Update

UPDATE Person
SET Person_Fname = 'hk'
WHERE Person_Id = 1;

SELECT * FROM Person;
```

Person_Id	Person_Fname	Person_Lname	Person_Email
1	Harsh	L-1	FL-1.@gmail.com
2	F-2	L-2	FL-2.@gmail.com
3	F-3	L-3	FL-3.@gmail.com
500	F-500	L-500	FL-500.@gmail.com

```sql
--here hk not update because INSTEAD OF fire
```

```sql
--INSTEAD OF Trigger use for DELETE
CREATE TRIGGER TR_Person_INSTEAD_OF_DELETE
ON Person
INSTEAD OF DELETE
AS
BEGIN

    SET NOCOUNT ON;

    DELETE FROM Person WHERE Person_Id = 3;
END
GO
```

```sql
SELECT * FROM Person;
```

Person_Id	Person_Fname	Person_Lname	Person_Email
1	Harsh	L-1	FL-1.@gmail.com
2	F-2	L-2	FL-2.@gmail.com
3	F-3	L-3	FL-3.@gmail.com
500	F-500	L-500	FL-500.@gmail.com

```sql
--normal delete

DELETE FROM Person WHERE Person_Id = 500;

SELECT * FROM Person;
```

Person_Id	Person_Fname	Person_Lname	Person_Email
1	Harsh	L-1	FL-1.@gmail.com
2	F-2	L-2	FL-2.@gmail.com
500	F-500	L-500	FL-500.@gmail.com

```sql
--here not delete 500 id
--because INSTEAD OF trigger using
```

Part: 4

```sql
--Step:6 new table create

CREATE TABLE Employee(
    Emp_Id INT,
    Emp_Fname VARCHAR(10),
    Emp_Lname VARCHAR(10),
    Emp_Email VARCHAR(20)
);
INSERT INTO Employee (Emp_Id,Emp_Fname,Emp_Lname,Emp_Email)
VALUES
    (1,'EF-1','LF-1','EF-1-@gmail.com'),
    (2,'EF-2','LF-2','EF-2-@gmail.com'),
    (3,'EF-3','LF-3','EF-3-@gmail.com'),
    (4,'EF-4','LF-4','EF-4-@gmail.com');

SELECT * FROM Employee;
```

Emp_Id	Emp_Fname	Emp_Lname	Emp_Email
1	EF-1	LF-1	EF-1-@gmail.com
2	EF-2	LF-2	EF-2-@gmail.com
3	EF-3	LF-3	EF-3-@gmail.com
4	EF-4	LF-4	EF-4-@gmail.com

```sql
--After Trigger for Insert

CREATE TRIGGER TR_Employee_After_Insert
ON Employee
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    PRINT 'You Can Not Insert Data..';
    ROLLBACK TRANSACTION;

END
GO
```

```sql
SELECT * FROM Employee;
```

Emp_Id	Emp_Fname	Emp_Lname	Emp_Email
1	EF-1	LF-1	EF-1-@gmail.com
2	EF-2	LF-2	EF-2-@gmail.com
3	EF-3	LF-3	EF-3-@gmail.com
4	EF-4	LF-4	EF-4-@gmail.com

```sql
INSERT INTO Employee (Emp_Id,Emp_Fname,Emp_Lname,Emp_Email)
VALUES
    (1000,'EF-1000','LF-1000','EF-1000-@gmail.com');
```

```
You Can Not Insert Data..
Msg 3609, Level 16, State 1, Line 134
The transaction ended in the trigger. The batch has been aborted.
```

```sql
--After Trigger for Update

CREATE TRIGGER TR_Employee_After_Update
ON Employee
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    PRINT 'You Can Not Update Data..';
    ROLLBACK TRANSACTION;

END
GO
```

```sql
SELECT * FROM Employee;
```

Emp_Id	Emp_Fname	Emp_Lname	Emp_Email
1	EF-1	LF-1	EF-1-@gmail.com
2	EF-2	LF-2	EF-2-@gmail.com
3	EF-3	LF-3	EF-3-@gmail.com
4	EF-4	LF-4	EF-4-@gmail.com

```sql
UPDATE Employee
SET Emp_Fname = 'harsh'
WHERE Emp_Id = 4;
```

```
You Can Not Update Data..
Msg 3609, Level 16, State 1, Line 157
The transaction ended in the trigger. The batch has been aborted.
```

```sql
SELECT * FROM Employee;
```

Emp_Id	Emp_Fname	Emp_Lname	Emp_Email
1	EF-1	LF-1	EF-1-@gmail.com
2	EF-2	LF-2	EF-2-@gmail.com
3	EF-3	LF-3	EF-3-@gmail.com
4	EF-4	LF-4	EF-4-@gmail.com

```sql
--After Trigger for Delete

CREATE TRIGGER TR_Employee_After_Delete
ON Employee
AFTER Delete
AS
BEGIN
    SET NOCOUNT ON;

    PRINT 'You Can Not Delete Data..';
    ROLLBACK TRANSACTION;

END
GO
```

```sql
SELECT * FROM Employee;
```

Emp_Id	Emp_Fname	Emp_Lname	Emp_Email
1	EF-1	LF-1	EF-1-@gmail.com
2	EF-2	LF-2	EF-2-@gmail.com
3	EF-3	LF-3	EF-3-@gmail.com
4	EF-4	LF-4	EF-4-@gmail.com

```sql
DELETE FROM Employee WHERE Emp_Id = 1;
```

```
You Can Not Delete Data..
Msg 3609, Level 16, State 1, Line 181
The transaction ended in the trigger. The batch has been aborted.
```

```sql
SELECT * FROM Employee;
```

Emp_Id	Emp_Fname	Emp_Lname	Emp_Email
1	EF-1	LF-1	EF-1-@gmail.com
2	EF-2	LF-2	EF-2-@gmail.com
3	EF-3	LF-3	EF-3-@gmail.com
4	EF-4	LF-4	EF-4-@gmail.com

- HK_Trigger
  - Database Diagrams
  - Tables
    - System Tables
    - FileTables
    - External Tables
    - Graph Tables
    - dbo.Employee
      - Columns
      - Keys
      - Constraints
      - Triggers
        - TR_Employee_After_Delete
        - TR_Employee_After_Insert
        - TR_Employee_After_Update
      - Indexes
      - Statistics
    - dbo.Person
      - Columns
      - Keys
      - Constraints
      - Triggers
        - TR_Person_INSTEAD_OF_DELETE
        - TR_Person_INSTEAD_OF_INSERT
        - TR_Person_INSTEAD_OF_UPDATE